

AI-NATIVE TECH ARCHITECT PROGRAMME

Capstone Case Study 1

Build Track

Build a Working Claude Code Development Harness

Track	Build (Solution Architects, Senior Architects)
Effort	~20–24 hours across 2–3 weeks
Tooling	Claude Code (claude.ai/code)
Stack	Learner's choice — Node.js / Java / Python / .NET (C#) or own non-proprietary project
Deliverable	Working harness repository + Harness Design Brief (3–5 pages) + demonstration run artefacts

Table of Contents

1. Purpose

This case study is the capstone project for solution architects and senior architects in the AI-Native Tech Architect programme at Cognizant. It assesses your ability to design and build a production-quality agentic harness using Claude Code — translating architectural intent into working software and documented design decisions that accelerate delivery whilst maintaining standards, quality, and governance.

The focus is harness engineering: orchestrating AI agents with clear responsibilities, structured handoffs, and measurable evaluation criteria. This is not a theoretical exercise — you will produce a working harness that runs autonomously on a provided reference project (or your own equivalent).

Programme note: This is the Build Track of Case Study 1. Build Track learners produce working code and a Design Brief; Design Track learners produce a Harness Architecture Design Document.

Why harness engineering?

AI coding tools are available in every client engagement. The gap is not access — it is structure. Without architectural guardrails, AI-generated code drifts from project standards, accumulates technical debt, and requires heavy human review. A well-designed harness closes this gap: it wraps AI code generation in a governed orchestration layer that enforces standards, provides feedforward context to each agent, and makes every AI-generated contribution production-ready.

For solution architects and senior architects, the harness skill has a second dimension: the ability to design and document governance systems clearly enough that engineering teams can implement and maintain them. This capstone assesses both dimensions — the working implementation and the architectural thinking behind it.

2. Context and Background

The client context

You are a Solution Architect at Cognizant, embedded with a retail client whose eight-developer squad is building the StoreOps API. The client's CTO has approved AI-assisted development using Claude Code. The engineering standards team has concerns: in a prior experiment, AI-generated code produced four specific failure modes:

1. Direct imports from another module's repository, bypassing the agreed service boundary and event bus
2. Raw Error throws in service methods, bypassing the project's typed `AppError` hierarchy
3. Tests that asserted HTTP status codes but did not verify business rule compliance
4. Missing event bus integration — state changes written directly to sibling module repositories

The standards team will not support the Claude Code rollout without a harness that prevents these failure modes from reaching main. You are building that harness.

Software delivery teams face growing pressure to ship faster without sacrificing quality, security, or architectural standards. AI coding tools like Claude Code are increasingly available, but without structure they produce inconsistent output that drifts from project standards and requires heavy human review.

A development harness solves this by wrapping AI code generation in a structured orchestration layer: agents with defined roles, skill files that provide feedforward context before each agent acts, and an evaluation loop that applies checks before output is accepted. The result is governed, standards-driven, AI-accelerated development.

2.1 The Harness

The harness has four components: Planner, Generator, Evaluator, and Monitor. Once the developer approves the Planner's spec, the Generator and Evaluator loop runs autonomously — the developer does not manually trigger each iteration. Claude Code drives this loop, with the developer re-entering only at escalation points. The Monitor records outcomes after every sprint run, providing the observability data needed to detect skill file drift and tune the harness over time.

Component	Role	What It Produces
Planner	Decomposes requirements	spec.md + sprint contracts with GIVEN/WHEN/THEN acceptance criteria
Generator	Implements sprint contracts	Code, tests, and generator-summary.md (AC self-check table, files changed, known gaps)
Evaluator	Reviews Generator output	Structured verdict (PASS / CONDITIONAL PASS / FAIL) with file-level and line-level feedback in evaluator-feedback.md
Monitor	Records sprint run outcomes for observability and skill file improvement	run-log.md per sprint: iterations used, verdict, escalation flag, estimated token cost, and quality trend notes — archived to .harness/reviews/

CLAUDE.md is the root orchestrator. It reads the Evaluator's verdict and decides the next action: advance to the next sprint on PASS, send feedback back to the Generator on FAIL, or escalate to the developer after a maximum of three iterations. The Monitor runs after each sprint verdict — it reads evaluator-feedback.md and writes a structured run-log.md to .harness/reviews/. This archive is the governance audit trail and the primary input for detecting which skill files need refinement.

2.2 How Claude Code Runs the Harness

Claude Code invocation model

Claude Code reads CLAUDE.md automatically when launched in your repository. Your CLAUDE.md defines the agents (by referencing the .harness/agents/*.agent.md files), the orchestration sequence, and the routing logic.

The Planner is invoked with a single developer prompt (e.g., '@planner Add shift handover bulk update to tasks'). Claude Code then drives the Generator/Evaluator loop autonomously, writing and reading handoff files in .harness/output/ to pass context between agents.

The developer's only active steps are: (1) invoke the Planner with a feature prompt, and (2) type APPROVED after reviewing spec.md. After that, Claude Code runs the loop until all sprints PASS or an escalation occurs.

Note: .harness/ is used instead of .github/ to keep harness files separate from CI/CD configuration. This is a deliberate structural choice — not a difference from the reference harness.

2.3 Setting Up Your Project

You do not need an existing codebase. You will generate the StoreOps project using Claude Code before building your harness. This gives every learner a consistent, well-specified starting point and eliminates concerns about proprietary or client code.

The full specification — architecture, module boundaries, coding standards, and API design — is in Section 3. Follow the four steps below to have your codebase ready before Day 1 of the effort plan.

Why generate the codebase first?

Generating StoreOps is itself a Claude Code exercise. You will see how Claude Code responds to a detailed specification prompt, inspect the output for quality, and correct any deviations before building the harness. This experience directly informs your skill file writing — you will know exactly what Claude Code gets right and where it needs guardrails.

Step 1 — Choose your stack and install prerequisites

Select one of the four supported stacks from Section 3.2. Ensure the following are installed before proceeding:

- Git (initialise a new local repository)
- Claude Code CLI — install via: `npm install -g @anthropic-ai/claude-code`
- Your stack runtime: Node.js 20 LTS / Java 17+ / Python 3.11+ / .NET 8 SDK
- Your stack toolchain: npm / Maven or Gradle / pip + venv / dotnet CLI

Step 2 — Create your repository

Create a new empty Git repository. This repository will hold both the generated StoreOps source code and your `.harness/` folder. Do not use a fork of an existing project.

Recommended repository name: **storeops-harness** (or `storeops-harness-java` / `storeops-harness-python` / `storeops-harness-dotnet` for non-Node stacks).

Step 3 — Generate the StoreOps codebase

Open the repository in Claude Code and use the bootstrap prompt below. Copy it exactly, replacing `[STACK]` with your chosen stack details from Section 3.3 and `[DOMAIN]` with your chosen domain context from Section 3.2.

StoreOps Bootstrap Prompt — paste into Claude Code

Generate the StoreOps REST API project using `[STACK]`.

Requirements:

- Domain: retail store operations management
- 5 modules: activities (operational activities), programmes (store programmes), staff (store staff), alerts (ops alerts), reports (store/region metrics)
- Each module: 3 layers (Routes → Service → Repository)
- Module boundary rules: no circular imports; notifications via event bus only; users is read-only for other modules
- Typed error hierarchy (AppError base class with code, message, statusCode) — no raw Error throws in services/routes
- 9 REST endpoints as specified in Section 3.6
- Test setup: `[Jest+supertest / JUnit5+MockMvc / pytest+httpx]`
- Linting/type checking configured and passing

Generate stub implementations (not full business logic) — enough structure to compile, run tests, and pass lint.

Use in-memory storage (no database required).

After Claude Code generates the code, review the output against the StoreOps specification in Section 3. If any module boundary rule, layer separation, or error-handling pattern is wrong, correct it before proceeding — the quality of your codebase directly determines the quality of your skill files.

Step 4 — Verify and commit the baseline

Run the automated checks for your stack before committing. All checks must pass:

Stack	Verification command	Expected result
Node.js	<code>tsc --noEmit && eslint . && jest --coverage</code>	0 type errors, 0 lint errors, all tests pass
Java	<code>mvn test</code> (or <code>./gradlew test</code>)	BUILD SUCCESS, all tests pass

Stack	Verification command	Expected result
Python	<code>mypy . && ruff check . && pytest --cov</code>	0 type errors, 0 lint errors, all tests pass
.NET (C#)	<code>dotnet build && dotnet test</code>	Build succeeded, 0 warnings (nullable on), all tests pass

Once all checks pass, start the application locally and verify at least one endpoint responds before committing:

Stack	Start command	Verify
Node.js	<code>npm start</code>	GET <code>http://localhost:3000/api/tasks</code> → 200 OK
Java	<code>./mvnw spring-boot:run</code> (or <code>./gradlew bootRun</code>)	GET <code>http://localhost:8080/api/tasks</code> → 200 OK
Python	<code>uvicorn app.main:app --reload</code>	GET <code>http://localhost:8000/api/tasks</code> → 200 OK
.NET (C#)	<code>dotnet run --project src/StoreOps.Api</code>	GET <code>http://localhost:5000/api/tasks</code> → 200 OK

Once the application is running and responding, make an initial commit labelled "baseline: generated StoreOps scaffold". This is your starting point — every change made by the Generator during your demonstration run will be visible against this baseline.

If the generated code does not pass checks

Iterate with Claude Code — describe the specific failure and ask it to fix the issue. Keep track of the corrections you make. These corrections are valuable material for your skill files: they tell you exactly what your Evaluator needs to check and what hard gates to set.

3. What You Will Build

This section defines your project: the application your harness will govern, the technology stack you will use, and the domain modules your skill files must encode. Read this section before writing a single skill file or agent definition.

3.1 The Reference Application: StoreOps API

StoreOps is a REST API for retail store operations management. It allows store teams to create operational programmes, assign and track activities across departments, coordinate staff, and surface performance reports by store and region. It is deliberately scoped to a single retail domain — complex enough to require architectural discipline, grounded enough that every business rule your skill files encode has a real-world counterpart.

You will generate StoreOps from scratch using Claude Code, get it running locally, build a harness that governs further development, and deploy the application to a cloud platform (AWS, Azure, or GCP). The end result is a live, deployable retail operations service whose feature additions were governed end-to-end by your harness.

Application	StoreOps — Retail Store Operations Management REST API
Domain	Retail store operations: operational activities, store programmes, staff coordination, performance reporting
Modules	activities, programmes, staff, alerts, reports (5 domain modules)
Architecture	3-layer model per module: Routes → Service → Repository. Cross-module via events.
Purpose	Capstone reference codebase — do not use in production

3.2 Choose Your Technology Stack

Select one of the four supported stacks. The StoreOps module structure, domain rules, and architectural constraints are the same regardless of stack — only the language, framework, and tooling differ. The programme provides a scaffold starter and coding standards for each option.

	Node.js (Option A)	Java (Option B)	Python (Option C)	.NET (Option D)	Own Project
Language	TypeScript 5.x (strict)	Java 17+ (or 21 LTS)	Python 3.11+	C# 12 / .NET 8	Your choice
Framework	Express 4.x	Spring Boot 3.x	FastAPI	ASP.NET Core Web API	Your choice
Testing	Jest + supertest	JUnit 5 + MockMvc	pytest + httpx	xUnit + WebApplicationFactory	Equivalent required
Linting / Types	ESLint + @typescript-eslint	Checkstyle + SpotBugs	pylint / ruff + mypy	StyleCop + nullable annotations	Equivalent required

	Node.js (Option A)	Java (Option B)	Python (Option C)	.NET (Option D)	Own Project
Auto check command	tsc --noEmit && eslint && jest -- coverage	mvn test (or ./gradlew test)	mypy . && ruff check && pytest -- cov	dotnet build && dotnet test	Define in your harness
Scaffold provided	Full scaffold + coding standards in programme materials	Starter project + standards guide in programme materials	Starter project + standards guide in programme materials	Starter project + standards guide in programme materials	Bring your own (non- proprietary)

Which stack should I choose?

Choose the stack you know best. Your skill files will only be credible if you can encode real decisions — naming rules, error-handling contracts, test patterns. A superficial skill file written in an unfamiliar language will score poorly. The Node.js scaffold is the most detailed reference implementation; choose it if you are comfortable with TypeScript. The .NET option suits architects working in Microsoft enterprise environments.

3.3 Module Structure

StoreOps is organised into five domain modules, each reflecting a distinct concern in retail store operations. Each module owns its own routes, service, repository interface, and types. This structure is consistent across all four supported stacks — the names and responsibilities are the same; only the file extensions and framework idioms differ.

Module	Retail responsibility	Key types / entities
activities	Manage operational activities — restocking runs, planogram resets, compliance checks, and general store tasks	Task, TaskStatus (TODO IN_PROGRESS DONE BLOCKED), TaskPriority (LOW MEDIUM HIGH CRITICAL), TaskCategory (RESTOCKING PLANOGRAM AUDIT COMPLIANCE GENERAL)
programmes	Manage store programmes and their staff membership — seasonal rollouts, compliance drives, store refits	Project, ProjectMember, ProjectRole (STORE_MANAGER DEPARTMENT_LEAD ASSOCIATE)
staff	Store staff registration, authentication, and profile management	User, UserProfile, StaffRole (REGIONAL_MANAGER STORE_MANAGER DEPARTMENT_LEAD ASSOCIATE), AuthToken
alerts	Deliver in-app alerts triggered by operational events — inventory	Notification, NotificationChannel (IN_APP EMAIL), NotificationStatus, AlertType

Module	Retail responsibility	Key types / entities
	flags, SLA breaches, and shift handover prompts	(INVENTORY SLA_BREACH SHIFT_HANOVER ESCALATION)
reports	Generate store and regional performance summaries — task completion rates, overdue counts by department, regional rollups on demand	Report, ReportType (STORE_SUMMARY REGIONAL_ROLLUP DEPARTMENT_PERFORMANCE), ReportStatus (PENDING READY FAILED)

Each module follows a strict three-layer model: Routes (HTTP + validation) → Service (business logic) → Repository (data access). Cross-module communication follows two rules: modules may call another module's service layer for read-only lookups; event-driven side effects (e.g., firing an SLA_BREACH notification when a CRITICAL task becomes overdue, or triggering a STORE_SUMMARY report when a programme closes) are raised via an event bus, never by direct service-to-service import. The reports module is read-only — it aggregates data across activities, programmes, and staff but never writes to those modules.

Full architecture specification

The detailed module boundary rules, error-handling contract, testing coverage thresholds, and API endpoint inventory are in Sections 3.4–3.6 of this document. Your skill files must encode these specifics, not generic principles — adapt naming and file-layout conventions to your chosen stack's idioms.

3.4 What Your Harness Must Govern

Your harness will be run against the StoreOps codebase. During the demonstration run, you will invoke the Planner with a retail operations feature prompt — asking the harness to add a new capability to StoreOps. Suggested features:

- Add shift handover bulk update — PATCH /api/activities/bulk-status allowing outgoing shift staff to mark multiple operational activities as DONE or BLOCKED in a single request, with partial failure handling and an audit entry per updated task
- Add SLA breach alerting — when a HIGH or CRITICAL task passes its due date without reaching DONE, automatically fire a SLA_BREACH notification to the assigned Department Lead and escalate to STORE_MANAGER if unresolved after a configurable grace period
- Add regional rollup report — GET /api/reports/region/:id aggregating task completion rates, overdue counts by TaskCategory, and blocked task lists across all stores in the region; triggers a REGIONAL_ROLLUP Report record via the event bus
- Add planogram task template — POST /api/programmes/:id/templates to clone a standard set of PLANOGRAM tasks into a new store programme, applying department assignments and default priorities from the template definition

The harness should generate **correctly layered** new code (routes → service → repository), **tests** that meet coverage thresholds, and pass **all automated checks** without manual intervention. After the harness run, the new feature must be demonstrable through the **running application** — call the new endpoint via curl or Postman and show a successful response.

Deployment target

You must deploy the application as part of your submission. Accepted deployment options:

- Local Docker — containerise the application (Dockerfile provided in scaffold) and run via docker compose up. Screenshot the running container and a successful API call.
- Cloud deployment — deploy to AWS (Elastic Beanstalk / ECS), Azure (App Service / Container Apps), or GCP (Cloud Run). Provide a live URL and demonstrate the new endpoint responding.

Cloud deployment is recommended for the AI Platform Services competency — it requires real decisions about environment configuration, secrets management, and service connectivity.

3.5 Architecture Rules Your Evaluator Must Enforce

These are the non-negotiable StoreOps architecture rules. Your evaluation hard gates and skill files must encode these specifically — not as generic principles. Each rule maps directly to one of the four failure modes identified in the client context (Section 2).

Rule	What it prohibits	How your Evaluator checks it
Module boundary	No module may import directly from another module's repository. Cross-module reads go through the target module's service layer only.	Automated: depcruiser or equivalent dependency analyser — zero cross-module repository imports
Event bus only	Side effects that cross module boundaries must be raised via the event bus — never by direct service-to-service import.	LLM-assessed: Evaluator checks that EventBus.emit() is used for cross-module triggers — no direct NotificationService or ReportService imports
Error contract	No raw Error throws in services or routes. All errors must use the AppError typed hierarchy (code, message, statusCode).	Automated (linter rule) + LLM-assessed: zero raw throw new Error() in services or routes
Layer separation	Routes → Service → Repository — no skipping. Routes must not contain business logic; repositories must not call external services.	LLM-assessed: Generator summary references correct layer for each changed file; Evaluator checks no HTTP logic in repositories

Rule	What it prohibits	How your Evaluator checks it
Read-only reports	The reports module aggregates data from activities, programmes, and staff — it never writes to those modules.	LLM-assessed: Evaluator verifies no write operations originate from the reports module

3.6 Base API Surface and Testing Thresholds

The nine base endpoints below are the starting REST surface your generated StoreOps codebase must implement, mapped to the activities, programmes, and alerts modules from Section 3.3. staff is auth-only (no CRUD surface here) and reports is added later as a demonstration feature (Section 3.4) — this mirrors the standalone/read-only treatment those two modules get throughout this document.

Method + Path	Module	Description
GET /api/activities	activities	List activities (optional programme and status filters)
POST /api/activities	activities	Create a new activity
GET /api/activities/:id	activities	Get activity by ID
PATCH /api/activities/:id	activities	Update activity fields (status, priority, category, assignee)
DELETE /api/activities/:id	activities	Delete an activity (owner or store manager only)
GET /api/programmes	programmes	List programmes for the authenticated store
POST /api/programmes	programmes	Create a new programme
POST /api/programmes/:id/members	programmes	Add a staff member to a programme
GET /api/alerts	alerts	Get alerts for the authenticated user

Minimum automated test coverage thresholds, applied regardless of stack:

Scope	Coverage Threshold
Service layer	80% line coverage minimum
Route / controller layer	70% line coverage minimum
Shared utilities	60% line coverage minimum
Overall project	70% line coverage minimum

4. Learning Objectives

On completing this capstone you will have demonstrated:

Competency	What You Demonstrate
Intent Specification Architecture	Translate feature requirements into structured intent — decomposed across agents with GIVEN/WHEN/THEN acceptance criteria, routing logic, and escalation paths documented in CLAUDE.md and agent files
Non-Deterministic Systems Design	Design an Evaluator whose verdict rules convert non-deterministic LLM output into a deterministic accept/reject decision — through hard gates, threshold-based checks, fallback paths, and an explicit escalation output when the iteration limit is exceeded
Architecture Governance	Encode project standards into skill files; enforce compliance via automated and LLM-assessed checks; maintain the sprint review archive as an audit trail of every acceptance decision
AI Platform Services — Claude	Operate Claude Code as a platform: agent configuration via CLAUDE.md, context window scoping per agent invocation, cost-aware skill file design (depth vs. token budget), and context reset strategy across sprints
Enterprise Architecture	Design a coherent multi-agent system with defined boundaries, explicit handoff contracts, and integration with the project's source structure, version control, and toolchain

5. Harness Components to Build

Your harness must be a working implementation in a Git repository. All components below are required.

5.1 CLAUDE.md — Orchestrator

The root CLAUDE.md is the orchestration brain of the harness. It must define:

- The entry prompt format a developer uses to start a harness run
- The agent files it references and how each is invoked
- The sequence: Planner → spec.md produced → AWAITING APPROVAL marker → developer types APPROVED → Generator/Evaluator loop begins
- Routing logic: how the orchestrator reads the verdict in evaluator-feedback.md and decides next action (next sprint, retry Generator, or escalate)
- The maximum iteration count per sprint before escalating — and what the escalation output looks like (e.g., a written escalation notice in .harness/output/escalation.md naming the sprint, iteration count, and blocking issue)
- Context scoping strategy: how context is reset or bounded between agent invocations to prevent context window degradation across long runs
- CI/CD relationship: a note on how the Evaluator's automated checks (lint, type check, test runner) relate to existing pipeline gates — does the harness precede, replace, or feed into CI checks

5.2 Agent Files

Three agent definition files in .harness/agents/. Each specifies the agent's responsibility, the skill files it reads before acting, and the handoff file it writes.

File	Reads (minimum)	Produces
planner.agent.md	app-context, architecture-principles, sprint-decomposition	spec.md (with STATUS: AWAITING APPROVAL marker), sprint-N-contract.md
generator.agent.md	app-context, architecture-principles + stack-specific skills	Code in src/, generator-summary.md (AC self-check table, files changed, known gaps)
evaluator.agent.md	architecture-principles, how-to-review + evaluation criteria skills	evaluator-feedback.md with structured verdict, per-check results, and file+line feedback on failures
monitor.agent.md	app-context (reads evaluator-feedback.md + generator-summary.md from completed sprint)	run-log.md archived to .harness/reviews/: sprint ID, verdict, iterations used, escalation flag, estimated token cost, quality trend notes

5.3 Skill Files (Minimum 6)

Skill files are feedforward guides — markdown files each agent reads before acting. You must produce at least 6, covering:

- Shared foundation (required for all agents): app-context, architecture-principles
- Generator-specific (minimum 2): e.g., coding-conventions, component-patterns, api-integration, how-to-test
- Evaluator-specific (minimum 2): e.g., how-to-review, grading criteria files

Skill files must reflect your actual project (StoreOps or your own). A generic file that could apply to any codebase will not pass evaluation. Each file should encode decisions specific to the project — module boundary rules, agreed error-handling contracts, test coverage thresholds, and stack-specific automated check commands.

Skill file format

Each skill file is a plain markdown file (SKILL.md) inside a named subdirectory under `.harness/skills/`. Structure it with a clear purpose statement at the top, followed by specific rules, examples, or reference material. Length should match the complexity of what the agent needs — a coding-conventions file for a typed language may run 2–4 pages; an app-context file may be shorter.

Sections 3.3–3.6 of this document map each StoreOps concept to the skill file that should encode it. Use that mapping as a checklist.

5.4 Evaluation Framework

The Evaluator is where your non-deterministic systems design competency is most directly tested. It must implement structured graded evaluation across at least two dimensions. Each dimension must define:

- A weight (all dimensions must sum to 100%)
- Hard gate conditions — specific failures that cause an immediate FAIL verdict regardless of other scores. These must be deterministic: given the same check result, the verdict is always the same.
- A checklist of specific checks with explicit pass/fail criteria
- How the Evaluator handles the inherent variability of LLM-generated code: what it checks, how it converts variable output into a binary per-check result, and what the fallback is when the Evaluator's own output is ambiguous

It is strongly recommended that at least one hard gate per dimension corresponds to an automated tool check — a linter, type checker, test runner, or dependency analyser appropriate to your stack (see the automated check commands in Section 3.2). Automated checks are deterministic by design and prevent verdict variability caused by LLM assessor leniency.

5.5 Demonstrated Run

You must demonstrate the harness running end-to-end on at least one feature from the list in Section 3.4 (or an equivalent feature of comparable complexity). Your submission must include:

- A PROMPT.md file in the repository root containing the feature prompt used to invoke the harness
- spec.md as produced by the Planner for that feature
- generator-summary.md for the demonstration sprint — committed to .harness/reviews/ alongside evaluator-feedback.md (provides the chain of evidence: contract → generator output → evaluator verdict)
- The sprint-1-contract.md (and any subsequent sprint contracts if the run included multiple sprints)
- At least one evaluator-feedback.md showing the verdict, per-check results, and reasoning — committed to .harness/reviews/
- The final code produced by the Generator in src/
- DEPLOYMENT.md: the deployment target chosen (local Docker or cloud), the steps taken, and a screenshot or URL showing the new endpoint responding in the running application
- REFLECTION.md (max 1 page): what the harness did well, where it fell short, and one concrete improvement you would make

6. Your Deliverables

This section defines exactly what you must submit and in what format. All four components are required. The Harness Design Brief is a new requirement for the SA-SM track — it is the document deliverable that demonstrates your architectural thinking alongside the working implementation.

Component	Weight	Format
Working harness (CLAUDE.md + agents + skills + eval framework)	45%	Git repository — see Section 5 for required components
Harness Design Brief	17%	DESIGN_BRIEF.md committed to repository root (Markdown, 3–5 pages) — see Section 6.2 for required content
Demonstration run artefacts	28%	Files committed to .harness/reviews/, DEPLOYMENT.md, REFLECTION.md — see Section 6.3
Architecture journal — JOURNAL.md (optional)	+10% bonus	JOURNAL.md in repository root — design decisions, trade-offs, and insights captured during the build

6.1 Working Harness Repository (45%)

Your working harness must be a complete implementation in a Git repository. The required components are defined in Section 5: CLAUDE.md orchestrator, three agent files, minimum six skill files, and a functional evaluation framework. The harness must run end-to-end on a StoreOps feature without manual intervention.

The repository structure (see Section 7) must follow the .harness/ layout with working files gitignored and sprint reviews committed to .harness/reviews/ as a permanent governance audit trail.

6.2 Harness Design Brief (17%)

The Harness Design Brief is a 3–5 page document that captures the architectural intent behind your working harness. It demonstrates your design thinking — how you decomposed intent, designed the governance layer, and reasoned about non-determinism. Diagrams count toward the page total.

The brief must cover four sections:

Section A — Intent Decomposition (0.5–1 page)

How you decomposed the chosen StoreOps feature into agent responsibilities and sprint contracts. Document:

- How the feature was broken into sprint contracts and why sprint boundaries were drawn where they are
- How acceptance criteria were structured using GIVEN/WHEN/THEN and what made each criterion testable rather than subjective
- One example sprint contract entry with its acceptance criterion in full

Section B — Governance Framework (1 page)

How your harness enforces StoreOps architectural standards. Document:

- Your skill file strategy: which files govern which aspects of the codebase, which are shared across agents and why, and what each file encodes that is specific to StoreOps (not generic)
- How the .harness/reviews/ archive functions as a governance audit trail — what is captured, who can access it, and how it would surface a recurring quality issue
- One skill file rule that encodes a specific StoreOps architecture decision (e.g., the event bus constraint, the AppError contract, or a module boundary rule) — state the rule and explain what breaks without it

Section C — Non-Determinism Strategy (1 page)

How your Evaluator converts inherently variable LLM output into deterministic accept/reject verdicts. Document:

- Your evaluation dimensions, their weights, and why you chose those dimensions for the StoreOps context
- Your hard gate conditions — for each hard gate, name the specific failure mode it prevents and why it cannot be a soft check
- How your verdict rules produce the same outcome given the same check results — walk through one example from variable Generator output to a definitive PASS or FAIL verdict
- Your escalation path: what triggers it, what the escalation output contains, and who receives it

Section D — Architectural Decisions (0.5–1 page)

Two or three key design decisions you made during the build. For each:

- The decision
- The alternatives you considered
- Your rationale
- An assumption the decision depends on

Why a Design Brief for a Build Track capstone?

A working harness without documented reasoning is a black box. As a solution architect or senior architect, your contribution is not just that the system works — it is that the system is understandable, maintainable, and defensible. The Design Brief tests whether you can articulate the architectural intent behind your implementation clearly enough that another architect could maintain, extend, or adapt the harness for a different team or codebase.

6.3 Demonstration Run Artefacts (28%)

You must demonstrate the harness running end-to-end on at least one feature from Section 3.4. Your submission must include all of the following files committed to the repository:

- **PROMPT.md** — the feature prompt used to invoke the harness
- **spec.md** — as produced by the Planner for that feature
- **sprint-1-contract.md** (and any subsequent sprint contracts if the run included multiple sprints)
- At least one **evaluator-feedback.md** showing the verdict, per-check results, and reasoning — committed to `.harness/reviews/`
- The final code produced by the Generator in `src/`
- **DEPLOYMENT.md** — deployment target (local Docker or cloud), steps taken, and a screenshot or URL showing the new endpoint responding in the running application
- **REFLECTION.md** (max 1 page) — what the harness did well, where it fell short, and one concrete improvement you would make

7. Repository Structure

Path	Purpose
CLAUDE.md	Root orchestrator — harness entry point, agent references, loop logic, escalation output definition
PROMPT.md	The feature prompt used for the demonstration run (committed to repo)
REFLECTION.md	Max 1-page reflection on the demonstration run
DESIGN_BRIEF.md	Harness Design Brief — 3–5 pages covering intent decomposition, governance framework, non-determinism strategy, and architectural decisions
JOURNAL.md (optional)	Architecture journal — design decisions, trade-offs, and insights captured during the build
.harness/agents/	planner.agent.md, generator.agent.md, evaluator.agent.md, monitor.agent.md
.harness/skills/	All skill files in named subdirectories (e.g., app-context/SKILL.md, how-to-review/SKILL.md)

Path	Purpose
.harness/output/	Working files during a run: spec.md, sprint-N-contract.md — gitignored during active run
.harness/reviews/	Archived sprint artefacts: evaluator-feedback.md, generator-summary.md, and run-log.md per sprint (e.g., sprint-1-evaluator-feedback.md, sprint-1-run-log.md) — committed as permanent quality and observability record
src/	StoreOps source code. Modules: activities/, programmes/, staff/, alerts/, reports/, shared/ (see scaffold)
tests/	Test files. For StoreOps: mirroring the module structure in src/

8. Assessment

8.1 Submission Components and Weights

Component	Weight	Format
Working harness (CLAUDE.md + agents + skills + eval framework)	45%	Git repository
Harness Design Brief (3–5 pages)	17%	DESIGN_BRIEF.md committed to repository root
Demonstration run artefacts	28%	Files committed to .harness/reviews/, DEPLOYMENT.md, REFLECTION.md
Architecture journal — JOURNAL.md (optional)	+10% bonus	JOURNAL.md in repository root

The maximum score without the optional architecture journal is 90%. Weights are distributed across all five programme competencies — see the rubrics below for the explicit mapping.

8.2 Evaluation Rubric — Working Harness (45%)

Criterion	Wt.	Competency	What Reviewers Look For
Intent specification — orchestration	9%	Intent Specification Architecture	CLAUDE.md defines entry format, agent references, routing logic, loop bounds, escalation output; Planner produces sprint contracts with GIVEN/WHEN/THEN acceptance criteria
Agent design and handoff contracts	9%	Intent Specification Architecture	Each agent has a clearly bounded, non-overlapping responsibility; skill reads listed and justified; handoff artefact format and machine-readable routing markers are explicit
Skill file quality and governance	9%	Architecture Governance	Files are StoreOps-specific, actionable, and encode real architectural decisions — not generic principles; .harness/reviews/ archive is structured as a governance audit trail; skill file rules are traceable to specific StoreOps constraints
Evaluation framework rigour	9%	Non-Deterministic Systems Design	Dimensions sum to 100%; hard gate conditions are stated and deterministic; at least one hard gate per dimension corresponds to an automated tool check; verdict rules produce the same outcome given the same check results
Monitor and observability	2%	Architecture Governance	monitor.agent.md is present and produces a structured run-log.md; at least one run-log.md is committed to .harness/reviews/ for the demonstration sprint; log captures sprint ID, verdict, iterations, escalation flag, and token cost estimate — sufficient to detect quality trends over time

Criterion	Wt.	Competency	What Reviewers Look For
Platform and context strategy	5%	AI Platform Services	Context window scoping per agent invocation is defined; context reset strategy across sprints is explicit; skill file depth reflects cost-awareness (depth vs. token budget trade-off is acknowledged)
Enterprise integration	2%	Enterprise Architecture	CLAUDE.md or Design Brief Section B addresses how Evaluator automated checks relate to existing pipeline gates (precede, replace, or feed into CI checks); harness files are correctly separated from CI/CD configuration in .harness/ vs .github/

8.3 Evaluation Rubric — Harness Design Brief (17%)

Criterion	Wt.	Competency	What Reviewers Look For
Intent decomposition quality	4%	Intent Specification Architecture	Agent responsibilities clearly derived from feature intent; sprint boundaries justified; acceptance criteria are testable with GIVEN/WHEN/THEN structure; example sprint contract entry demonstrates the pattern
Governance framework clarity	4%	Architecture Governance	Skill file strategy documented with StoreOps-specific reasoning; audit trail design explains how quality issues would be surfaced; at least one skill file rule is traceable to a specific StoreOps architectural constraint
Non-determinism strategy	6%	Non-Deterministic Systems Design	Evaluation dimensions and hard gate rationale explained; variable-to-deterministic conversion logic articulated for the StoreOps context; escalation path designed with specifics — who receives it, what it contains, expected response
Architectural decision log	3%	Enterprise Architecture	2–3 key decisions documented with options considered, rationale, and assumptions; decisions are specific to this harness design — not generic patterns restated

8.4 Evaluation Rubric — Demonstration Run (28%)

Criterion	Wt.	Competency	What Reviewers Look For
Harness ran end-to-end	10%	Intent Specification Architecture	All artefacts present and internally consistent: spec.md references the feature prompt, sprint contract references the spec, generator-summary references the contract's ACs, evaluator-feedback references specific files from the generator output

Criterion	Wt.	Competency	What Reviewers Look For
Evaluator quality	8%	Non-Deterministic Systems Design	Verdict is reasoned and specific — file names and line-level references are present; hard gates were applied correctly; feedback is actionable and traceable to the evaluation framework designed in the brief
Deployment evidence	4%	AI Platform Services	DEPLOYMENT.md present; deployment target is clear (Docker or cloud); screenshot or URL shows the harness-generated feature responding in the running application
Reflection depth	6%	Architecture Governance	Honest, specific assessment of harness strengths and limitations; improvement identified in REFLECTION.md is concrete and actionable; reflection connects back to a decision in the Design Brief

Note on AI tool use

You may use Claude Code or other AI tools to help draft skill files and agent definitions. The architectural decisions — what to include, how to structure the evaluation, what the hard gates are, how to scope context, and how to document the governance rationale in your Design Brief — must be yours. Reviewers will probe these decisions directly during evaluation.

9. Suggested Effort Plan

Before you start

1. Review Sections 3.3–3.6 of this document before Day 1. They provide the full architecture specification, module boundary rules, and base API surface for your chosen stack.
2. Read the programme reference harness (available in programme materials). It demonstrates the Planner/Generator/Evaluator pattern with concrete examples of skill files, agent definitions, handoff file formats, and verdict rules.
3. Familiarity with both documents before Day 1 will make your effort estimates accurate and your skill files credible.

Day	Phase	Activity	Effort
1	Project setup + generation	Review Sections 3.3–3.6 + reference harness; generate StoreOps codebase using Claude Code (Section 2.3 steps); verify checks pass; commit baseline; sketch agent responsibilities	2-3 hrs
2-3	Planner build	Write sprint-decomposition skill; build planner.agent.md; test on a retail feature prompt from Section 3.4; review spec.md output for completeness and AC quality	2-3 hrs
4-6	Generator + skills	Write stack-specific skill files (coding-conventions, api-integration, how-to-test); build generator.agent.md; run on Sprint 1 contract; inspect generator-summary.md output	3-4 hrs
7-9	Evaluator build	Design evaluation dimensions, weights, hard gates, and automated checks (use the stack auto check command from Section 3.2); write how-to-review and criteria skill files; build evaluator.agent.md; verify verdict determinism; build monitor.agent.md and verify run-log.md output	3-4 hrs
10-11	Full harness run + Design Brief A–B	Run the full harness end-to-end on your chosen retail feature; debug until all sprints PASS or escalation is documented; capture artefacts in .harness/reviews/; draft Design Brief Sections A (intent decomposition) and B (governance framework) while the harness run is fresh	3-4 hrs
12-13	Deploy + Design Brief C–D	Deploy the application (local Docker — minimum; cloud — optional for AI Platform Services merit); verify the harness-	4-5 hrs

Day	Phase	Activity	Effort
		generated endpoint responds in the running app; screenshot or record live URL; write REFLECTION.md; complete Design Brief Sections C (non-determinism strategy) and D (architectural decisions)	
14	Final review	Review repository for consistency — all required files committed, DESIGN_BRIEF.md complete, repo structure matches Section 7; final self-check against the rubric	2-3 hrs

10. Quality Markers

Strong submissions share a common characteristic: every design decision is traceable to the StoreOps context and justified against the four failure modes in Section 2. A reviewer should be able to point to any element — a hard gate, a skill file rule, a Design Brief decision log entry — and immediately understand why it exists for this retail squad.

Strong submission	Weak submission
Hard gates name specific automated checks for the chosen stack — e.g., "dep cruiser must report zero cross-module repository imports" or "tsc --noEmit must return 0 errors" Skill files contain rules traceable to StoreOps — e.g., "EventBus.emit('SLA_BREACH', payload) is the only permitted cross-module trigger; direct import of NotificationService is a hard gate failure"	Hard gates are generic — "code quality must be acceptable" or "all tests must pass" with no specificity Skill files contain generic principles that could apply to any REST API without modification
Design Brief Section C (Non-Determinism Strategy) names the specific failure mode each hard gate prevents and explains why it cannot be a soft check Evaluator feedback in .harness/reviews/ names specific file paths, line numbers, and the rule violated — actionable enough that the Generator can fix without human clarification REFLECTION.md identifies a specific harness limitation observed during the demonstration run and proposes a concrete improvement (e.g., "the Evaluator passed a module boundary violation because dep cruiser was not configured for the alerts module — fix: add alerts to the dep cruiser config") Design Brief Section D decision log entries name the options considered and the assumption the chosen decision depends on — not just "I chose this because it was simpler"	Design Brief is a summary of the working harness with no additional architectural reasoning Evaluator feedback is vague — "code does not follow conventions" or "tests are insufficient" without specifics REFLECTION.md is generic ("the harness worked well but could be improved") with no specific observation from the actual run Decision log is a list of things done during the build, not a record of choices made between alternatives

11. Closing Note

A great submission is not the one with the most skill files -- it is the one where every file earns its place. Reviewers are looking for architectural judgment: you understand why each agent is scoped the way it is, why each hard gate exists, and what would break if it were removed.

The harness you build here is a practical artefact you can carry into client engagements and share with your team. Treat it accordingly.